Co-Compression via Superior Gene for Remote Sensing Scene Classification

Weiying Xie, *Member, IEEE*, Xiaoyi Fan, Xin Zhang, Yunsong Li, *Member, IEEE*, Min Sheng, *Senior Member, IEEE*, and Leyuan Fang, *Senior Member, IEEE*

***Abstract*— Convolutional neural networks (CNNs) have been successfully employed in remote sensing image classification because of their robust feature representation for different visual tasks and powerful graphics processing units (GPUs). The attendant problem is that high computational cost and high memory footprint hindering the application of CNNs for remote sensing applications in resource- and time-sensitive situations. Based on practical deployment requirements, we pioneer a pruning-quantization joint learning model compression method for remote sensing image classification, called co-compression via superior gene (CC-SG). An enhanced evolution algorithm (EEA) is adopted as the agent to search a "superior gene," and immediately following, a director receives the "superior gene" and gives a compression mask and a resource constraint feedback to the agent. The network is eventually compressed and fine-tuned according to the optimal compression mask. Specifically, we introduce gene age and progressive shrinkage mutation rate to EEA and design a fitness function that balances accuracy and resource constraints. As validated using the UC Merced land-use and NWPU-RESISC45 datasets, the proposed CC-SG demonstrated superiority over other compared model compression approaches. For example, CC-SG attained substantial bit operations (BOPs) compression ratio of 40.04 with 0.956% accuracy increase for VGG-16 on UC Merced land-use dataset and 40.00 with 0.203% accuracy increase for ResNet-56 on NWPU-RESISC45 dataset. The code is available at https://github.com/fanxxxxyi/CC-SG.**

***Index Terms*— Enhanced evolutionary algorithm, pruning-quantization joint learning, remote sensing scene classification.**

I. Introduction

REMOTE sensing image scene classification is employed to identify the land cover type and is of great significance in many remote sensing applications, such as environmental monitoring, urban planning, agriculture, forestry, and hydrology. Due to its wide range of usages, great efforts have been devoted to the development of various remote sensing image classification methods in the past decades [1], [2], [3]. Early methods typically endeavored to extract holistic and differentiated feature representations, such as bag-of-visual words (BoVW) [4], histogram of oriented gradients (HOG) [5], and scale-invariant feature transformation (SIFT) [6]. However, an apparent drawback of feature extraction is that those rely heavily on the experience of the researcher. Furthermore, with the increased spatial resolution in recent years, traditional frameworks have seen limitations in feature representation due to large changes in spatial arrangement and structural patterns.

Convolutional neural networks (CNNs) have been successfully adopted for remote sensing due to their robust feature representation capabilities for a diverse range of visual tasks and their powerful graphics processing units (GPUs) [7], [8], [9], [10]. Unlike shallow models and rudimentary methods that rely on manual feature extraction, deep convolutional networks can tackle more complex scenes and gain better classification performance by learning in-depth visual features at a denser abstraction from large training datasets. A concomitant problem is that public remote sensing image datasets are typically smaller than natural ones consisting of millions of samples, leading to substantial redundancy in parameters and network structure. More seriously, the high computational cost and high memory footprint required for over-parameterization hinder the application of remote sensing techniques in resource- and time-sensitive scenarios or on limited hardware endpoints, such as satellite in-orbit processing platforms, mobile platforms, real-time inference systems on embedded devices, and so on. In this context, a host of network compression techniques have been proposed, including compact network design [11], [12], tensor decomposition [13], [14], network pruning [15], [16], quantization [17], [18], and knowledge distillation [19], [20].

Both network pruning and quantization are favorable for deploying CNNs on resource-limited hardware. Network pruning yields a compact subnetwork by pruning weights (fine-grained) or pruning filters/channels (coarse-grained). Although the weight pruning approach will better maintain network accuracy, it results in unstructured sparsity in the filters. So, it cannot accelerate universal hardware, while hardware-friendly structured pruning is more widely used. Quantization converts costly floating-point operations (FLOPs) into hardware-friendly low-bit operations (BOPs) and can be classified according to the quantization bitwidth:

Manuscript received 27 January 2023; revised 16 February 2023; accepted 19 February 2023. Date of publication 22 February 2023; date of current version 1 March 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 62121001 and Grant U22B2014 and in part by the Young Elite Scientist Sponsorship Program by the China Association for Science and Technology under Grant 2020QNRC001. *(Corresponding author: Xiaoyi Fan.)*

Weiying Xie, Xiaoyi Fan, Xin Zhang, Yunsong Li, and Min Sheng are with the State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an 710071, China (e-mail: wyxie@xidian.edu.cn; fanxxxxyi_xdu@163.com; xinzhang_xd@163.com; ysli@mail.xidian.edu.cn; msheng@mail.xidian.edu.cn).

Leyuan Fang is with the College of Electrical and Information Engineering, Hunan University, Changsha 410082, China (e-mail: fangleyuan@gmail.com).

Digital Object Identifier 10.1109/TGRS.2023.3247872

1558-0644 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 22:05:33 UTC from IEEE Xplore. Restrictions apply.

binary, fixed-precision quantization, and mixed-precision quantization. As a rule, practical deployments usually require a combination of both, whereas most current practice [18], [21], [22], [23], [24] performed filter pruning and quantization alone or a simple overlay of several methods, such as [25]. In spite of the fact that the resulting compressed CNN is less resource intensive, it would lead to a suboptimal solution as the two are not learned together to maximize their respective advantages through cooperation. Apart from this, Guo et al. [25] only designed the VGG [26] without considering other generic networks, such as ResNet [27], who contains multiple residual blocks. AJPQ [28] tried to migrate the AutoML-based pruning-quantization integration framework for remote sensing object detection after learning natural image classification. Compared with artificial rule-based pruning methods, the AutoML-based scheme can automatically search for the best configuration, but the reinforcement learning used by AJPQ is not as fast as evolutionary algorithm, especially in the early stages [29]. Meanwhile, the limitation of computational resources for the search process is an issue to be considered.

In this article, we propose an integrated framework of structured pruning and quantization for remote sensing scene classification named co-compression via superior gene (CCSG), which transforms the co-compression strategy search into a neural network sparse structured space exploration problem from a global perspective. We perceive pruning as 0-bit filter-level quantization, so that independent pruning and quantization can be reduced to one-step mixed precision quantization. Alternatively, we consider that the "mixed precision quantization bitwidth" of each filter can be globally determined by "gene" corresponding to its layer given by the agent enhanced evolution algorithm (EEA). EEA, who is used to find "superior gene," is with candidate age and progressive shrinkage mutation rate and introduce a hardware resource constraint as a co-compression's optimization metric to act a fitness function.

Subsequently, director calculates each filter's score via the "gene" searched in each iteration and sorts these scores in global. In our approach, we assume that the more important a filter is, the wider the weight and activation quantization bitwidth it should maintain. Then, the director gives a compression mask in accordance with the global sorting and a resource constraint feedback to the agent. The agent updates the population after calculating the fitness function for the current "gene." When the highest fitness function in the population is "superior gene" at the end of the iteration, so the corresponding compression strategy is found and the network can then be compressed and fine-tuned.

In summary, our contributions are in three main areas.

1) We put forward a pruning-quantization co-compression framework to search for the globally optimal compression strategy while maintaining the performance of the compressed model. To the best of our knowledge, this is the first co-compression in the field of remote sensing scene classification using the AutoML approach. We aim to automatically find high-quality schemes within the constraints, without the need for any manual experience. Extensive experiments on benchmark datasets demonstrated the efficiency and effectiveness of our method.
2) We transform the co-compression strategy search into a neural network sparse structured space exploration problem, treating pruning as 0-bit filter-level quantization, so that independent pruning and quantization can be reduced to one-step mixed precision quantization.
3) We then put forward an EEA as the agent to find the "superior gene." EEA is with candidate age and progressive shrinkage mutation rate, bringing in hardware resource constraints and applying a co-compression's optimization metric as a fitness function.

The rest of this article is organized as follows. In Section II, we describe the proposed approach in detail. In Section III, a series of experiments on three popular networks and two different datasets are reported. A comprehensive evaluation confirms the effectiveness of our proposed method. Finally, the conclusions of the study are drawn in Section IV.

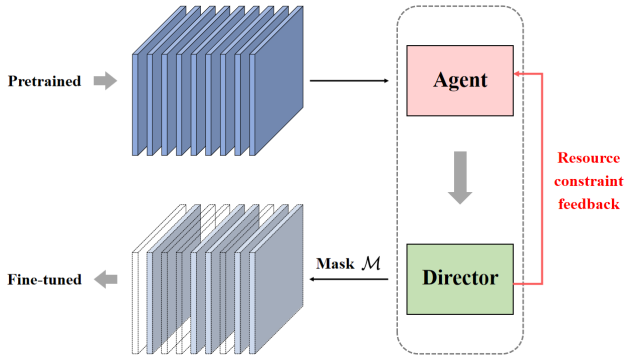

Fig. 1. Illustration of the proposed framework. After a full-precision over-parameterized CNN model is pretrained, EEA as an agent seeks for " genes," and a model compression strategy is generated by the director. Besides, in each iteration, the director gives the agent a resource constraint feedback. Finally, the model is fine-tuned after compression to obtain the final compressed CNN.

II. Methodology

The framework of the method is shown in Fig. 1. In general, a typical network compression process consists of three stages: 1) training the over-parameterized model; 2) compressing the large trained model according to certain guidelines; and 3) fine-tuning the compressed model to recover the lost performance. We basically follow this strategy, and our framework can be broadly divided into three procedures. As shown in Fig. 1, a full-precision over-parameterized CNN model is first trained on the target dataset; then, EEA as an agent seeks for "genes," and a model compression strategy is generated by the director. Besides, in each iteration, the director gives the agent a resource constraint feedback. Finally, the model is fine-tuned after compressed.

A. Description

Assume a CNN model with N convolutional layers, where the number of filters in each layer is i. l denotes the layer index, $1 \leq l \leq L$. i denotes the layer index, $1 \leq i \leq I$. Let $\mathcal{W}$ be the network weights, and $\mathcal{M}$ be

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 22:05:33 UTC from IEEE Xplore. Restrictions apply.

the compression strategy (including structured sparse and quantization strategies). $\Omega(\mathcal{M})$ refers to the computational complexity of the compression network that employs the compression strategy $\mathcal{M}$. The core objective of this research is to be able to balance both the computational complexity constraints and the accuracy of the compressed model, so the search objective function is written in the following bi-level optimization form when given a labeled dataset $D = (x, y)$

$$\begin{aligned} &\min_{\mathcal{M}} \mathcal{L}_{\text{val}}\left(\mathcal{W}^*(\mathcal{M}), x; y\right) \\ &\text{s.t. } \mathcal{W}^*(\mathcal{M}) = \arg\min \mathcal{L}_{\text{train}}(\mathcal{W} \odot \Phi(\mathcal{M}), x; y) \\ &\quad \Omega(\mathcal{M}) \le \Omega_0 \end{aligned} \tag{1}$$

where $\mathcal{L}_{\text{val}}$ and $\mathcal{L}_{\text{train}}$ depict the task loss on the validation data and the training data. $\mathcal{W}^*(\mathcal{M})$ indicates compressed weights. $\Phi(\mathcal{M})$ is a function that can allocate the compression strategy $\mathcal{M}$ to corresponding positions. Ω_0 stands for the resource constraint of the deployment platform.

B. Searching for Compression Strategy With the Help of Agent and Director

Previous work has shown that eliminating redundant, i.e., relatively unimportant filters and quantizing filter weights and activation functions in CNNs, can be done almost lossless [18], [21], [22], [30]. A widely used measurement in metric-based compression methods is the filter importance score, which can be calculated from the $L1$-norm, $L2$-norm, Rank [31], or other filter properties derived from filter weight or feature map. We break with such a tradition by neither using filter properties alone as the unique metric nor by using each layer sparsity or quantization bitwidth as the AutoML agent search object, such as [32].

More specifically, we use a mapping mechanism between pairs of affine transformations for each layer and the "genes" belonging to each layer to realize cross-level decision-making from a global perspective. The following definition is given first:

$$\mathcal{G} = \{\{\mathcal{A}_l, \mathcal{B}_l\} \mid 1 \le l \le L\} \tag{2}$$

where $\mathcal{G}$ is the set of "genes" we are looking for, containing pairs of affine transformation coefficients $\mathcal{A}_l, \mathcal{B}_l$ designed for each layer of the network.

In turn, depending on the $\mathcal{G}$ generated by each iteration of the agent's search, we can construct a score for each filter

$$\mathcal{S}_l^i = \mathcal{A}_l \times \Psi\left(\mathcal{W}_l^i\right) + \mathcal{B}_l \tag{3}$$

where $\Psi(\mathcal{W}_l^i)$ could be any filter property derived from filter weight or feature map, such as $L1$-norm, $L2$-norm, or Rank. Our selection criterion is that the chosen filter property measures the importance of the corresponding filter input to the CNN among all the filters in the same layer. Apart from the common filter properties listed, there are a number of alternative calculations proposed in other papers, such as geometric centers in FPGM [22] or channel independence in CHIP [33].

$\mathcal{S} = \{\mathcal{S}_l^i\}, 1 \le i \le I, 1 \le l \le L$ is then arranged in descending order to provide $\mathcal{S}_\downarrow$, who is passed through a director to produce a current compression mask $\mathcal{M}$, which corresponds to the compression strategy

$$\mathcal{M} = \text{Director}\left(\mathcal{S}_\downarrow\right). \tag{4}$$

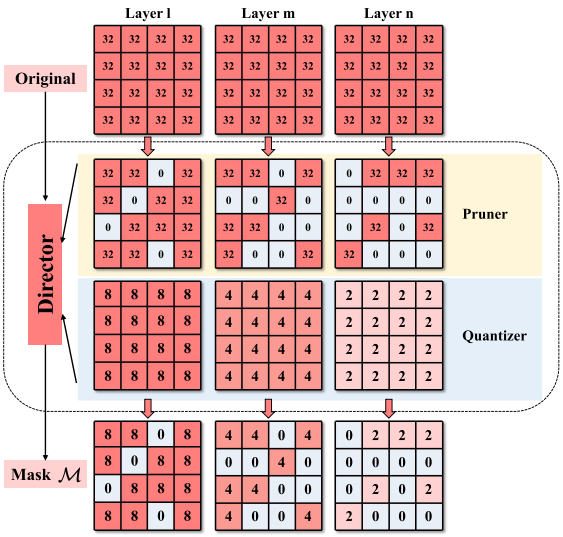

Fig. 2. Encoding process of the director who has a bit width selection of $\{0, 2, 4, 8\}$ bits. Specifically, the director can be divided into a pruner and a quantizer, which are responsible for selecting pruning objects and assigning each layer quantization bitwidths in the mixed precision quantization bitwidth assignment, respectively. Ultimately, the output masks of both are combined into one, and the bitwidth of each filter is the minimum bitwidth at the corresponding position.

The encoding process of the director is shown in Fig. 2. We can translate the optimization problem (1) into

$$\arg\max_{\mathcal{G}} \sum_{l=1}^{L} \sum_{i=1}^{I} \mathcal{S}_l^i \odot \mathcal{M}_l^i \tag{5}$$

where $\odot$ denotes the Hadamard product, meaning that the corresponding position elements are multiplied together.

As DJPQ [34] mentioned, only power-of-two bitwidths are often efficiently implemented and supported in typical digital hardware, and other bitwidths are rounded up to the nearest power-of-two values, resulting in inefficiencies in computation and storage. Following this idea, we limit our quantization bitwidth choices to $\{0, 2, 4, 8\}$. Of course, this will also reduce the search space.

Specifically, when the director assigns quantization bitwidths, it first determines the objects to be assigned 0 bit based on FLOPs and parametric constraints. For the remaining filters, two fixed thresholds of τ_1 and τ_2 are set on $\mathcal{S}_\downarrow$ score table, with score less than or equal to τ_1 assigned 2 bit, greater than τ_1 less than or equal to τ_2 assigned 4 bit, and greater than τ_2 assigned 8 bit (for convenience, filters on the same layer take the same bitwidth, i.e., the maximum of all bitwidths for that layer of filters).

As shown in Algorithm 1, after assigning the mixed precision quantization bitwidth, the director gives the mask and calculates the current hardware computations; if the current hardware computations is greater than the constraints, the threshold is reduced by $\Delta\tau$; then, the mask is given again, and the current hardware computations are calculated and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 22:05:33 UTC from IEEE Xplore. Restrictions apply.

judged until the current hardware computations are less than or equal to the constraints; then, the searched mask $\mathcal{M}$ and the resource constraints are fed back to the agent. The agent will compress the network and calculate the fitness function based on the feedback to complete the current round of update. Until the entire EEA ends up and a "superior gene" $\mathcal{G}^*$ is found, resulting in a final compression strategy $\mathcal{M}^*$, the compressed network is completely fine-tuned to restore its performance.

The "superior gene" contains two levels of definition. From the point of view of optimization, the "superior gene" is the solution to the optimization problem (5) that makes it optimal. From the point of view of network, the "superior gene" is a set of optimal affine transformations that give a visual representation of the most reasonable cross-layer importance relations of all filters in the network. So, we designed the agent and the director according to the definitions given. The agent is used to search for "superior gene," and the experimental results also show its effectiveness. The director works with the agent and gives it continuous feedback. Together they work to optimize our optimization goals. Therefore, we consider the "gene" found by agent and director to be the superior one.

C. Guided Modification Designs in EEA

1) Design of the Fitness Function: As advocated by DJPQ [34], actual hardware measurement is generally a poor indicator of the usefulness of these methods. There are many possible target devices, e.g., GPUs, tensor processing units (TPUs), several phone chips, the Internet of Things (IoT) devices, field programmable gate arrays (FPGAs), and dedicated silicon, each with its own quirks and trade-offs. The efficiency of networks also greatly depends on specific kernel implementations, and currently, many devices do not have multiple bitwidth kernels implemented to even compare in a live setting. Therefore, we follow [34] to choose a more general metric to measure the compression performance, namely BOPs count, assuming that computations can be done optimally in an ideal world.

When searching for viable solutions for both pruning and quantization, we need to have a combination of both in a sensible way. This is why we define the fitness function as follows:

$$\mathcal{L}_{\text{fitness}} = \mathcal{L}_{\text{CE}} + \beta \cdot |\Omega(\mathcal{M}) - \Omega_0| \tag{6}$$

where β is a factor that ensures the order of magnitude equivalence.

In each period, EEA draws S random samples from the overall population, each with uniform random replacement. $\mathcal{G}$ with the highest validation fitness among them is selected as the parent. A new architecture, called child, is constructed from the parent by mutation. Mutation results in simple and random modifications to the architecture.

2) Design of the Progressive Shrinkage Mutation Rate: Genetic mutation operations are performed by randomly changing the proportion of the gene. The mutation operation reintroduces genetic diversity into the population and effectively prevents the search falling into a local optimum solution that cannot be broken out. In a typical implementation of an evolutionary algorithm, the mutation rate (the probability of changing a gene property) is a fixed value and very minimal, such as [35]. Therefore, the new set of genes produced by mutation will not differ significantly from the original. We devised a dynamic progressive mutation rate $\mathcal{R}_{\text{mut}}$

$$\mathcal{R}_{\text{mut}} = 1 - \frac{1}{1 + e^{-\frac{\sqrt{\frac{S}{P}} \cdot t}{\alpha}}} + k \tag{7}$$

where t is the current period index and $1 \le t \le T$, T is the total period number, α and k are both adjustable constants, α adjusts the speed of contraction of the asymptotic curve, and k determines the asymptote of the contraction approximation.

The $\mathcal{R}_{\text{mut}} \times L$ convolutional layers are randomly selected, and the corresponding $\{\mathcal{A}_l, \mathcal{B}_l\}$ of the extracted layers is mutated in the following manner:

$$\begin{aligned} \mathcal{A}_l &:= \mathcal{A}_l \times \hat{\mathcal{A}}_l \hat{\mathcal{A}}_l \sim e^{N\left(0, \left(1 - \frac{t}{\lambda T}\right)^2\right)} \\ \mathcal{B}_l &:= \mathcal{B}_l + \hat{\mathcal{B}}_l \hat{\mathcal{B}}_l \sim N(0, \text{std}[\Psi(\mathcal{W}_l)]) \end{aligned} \tag{8}$$

where λ is a constant, and $\text{std}[\Psi(\mathcal{W}_l)]$ is the variance of $\Psi(\mathcal{W}_l)$.

In the early periods of the EEA, the degree of variation is increased by a higher mutation rate, which consequently expands the scope of the agent exploration. As the search process stabilizes, the search space is gradually reduced to speed up the exploration and improve the stability of the algorithm. In addition, the mutation rate is negatively correlated not only with the number of iterations but also with the number of samples sampled as a proportion of the population size. Therefore, compared with the traditional mutation rate design, the progressive shrinkage mutation rate is more conducive to improving the search efficiency and bringing high-quality variant genes.

3) Design of the Gene Age: In tournament selection, it is common to keep the population size at the initial value of P. This is typically accomplished by an additional step within each period: discarding (or killing) the worst model in a random S sample. This approach is called non-aging evolution. Borrowing ideas from [29], we prefer a new method called aging evolution shown in Fig. 3. EEA associates each gene with an age and killing the oldest gene in the population, i.e., removing the earliest gene produced from the population. This allows us to have more search space, and instead of amplifying the power of elite gene prematurely as in non-aging evolution that has the possibility of premature convergence to a local optimum.

D. Compression and Fine-Tuning

Once the mask $\mathcal{M}^*$ is obtained, the network can be compressed. Filters assigned to the 0 bit will be pruned directly. In addition, when filters are pruned, layers with fewer channels are created, and the remaining parameters of the unaffected channels are copied into the new layer. The kernel in the next layer corresponding to the pruned feature mapping is also removed. To avoid a situation that all filters on a particular layer are pruned, we set a safety region for each layer, i.e., the minimum number of filters that should be kept

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 22:05:33 UTC from IEEE Xplore. Restrictions apply.

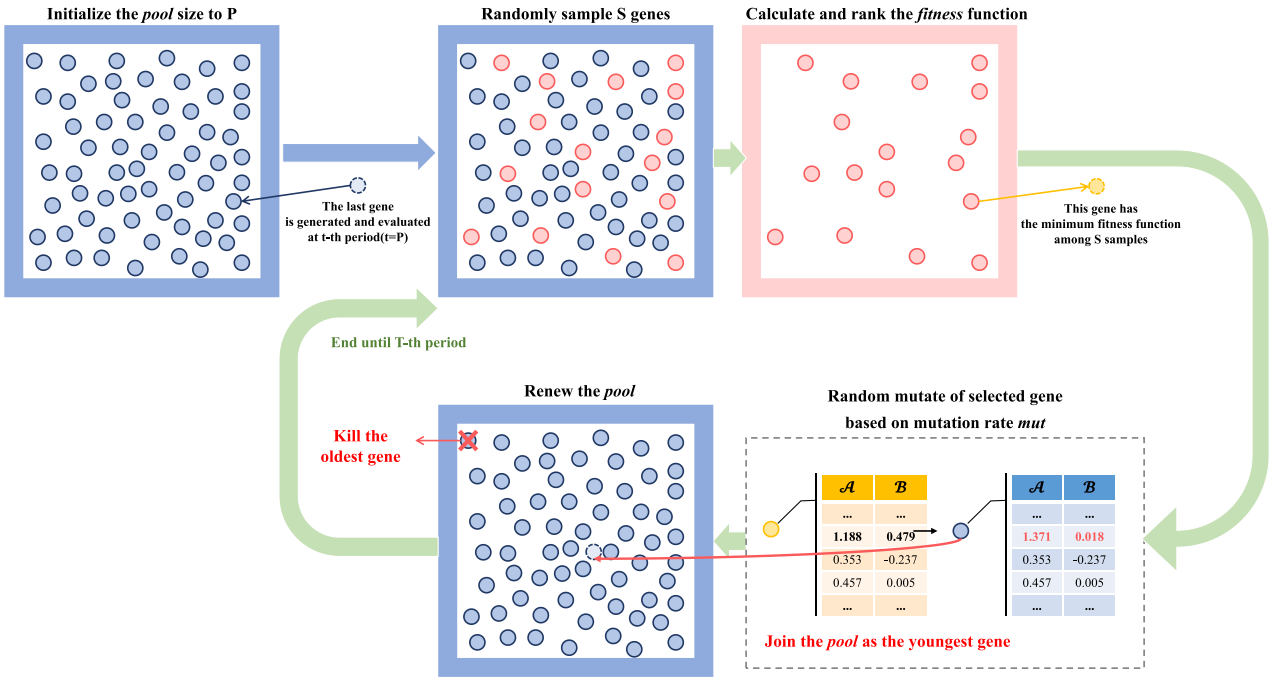

Fig. 3. Whole process of the agent, EEA. First, we initialize the pool size to P and randomly sample S genes from the pool. Then, we calculate and rank the fitness function among S samples. Next, we randomly mutate the selected gene based on the progressive shrinkage mutation rate and let the mutated gene join the pool as the youngest gene while killing the oldest one. This completes a round of pool renewal. Finally, we end until the Tth period and obtain the "superior gene".

in each layer. When the director recognizes that the remaining filters in the current layer are close to the safety region, it stops pruning and skips the layer.

For the remaining filters, the weight parameters W are quantified into B_W bit according to the following equation:

$$W^q = \frac{\text{round}\left[\frac{\tanh(W)2^{B_W-1}}{\max(|W|)}\right]}{2^{B_W-1}}. \tag{9}$$

The activation function is a B_A-bit quantization of the original full-precision activation function A

$$A^q = \frac{\text{round}\left[\text{clamp}(A,0,1)\cdot 2^{B_A}\right]}{2^{B_A}}. \tag{10}$$

In this way, a compression network with fewer parameters and lower inference costs is generated. After removing the channel or quantizing parameters, changes in the network structure and parameter may temporarily lead to some degree of performance degradation. Fine-tuning is a common method of compensating for the loss of accuracy in model compression. Even better performance can be achieved after fine-tuning due to the more logical architecture of the compressed network. Also, because the network is narrower, fine-tuning requires less runtime memory than the original training.

Algorithm 1 Algorithm Description of CC-SG

Input: Pretrained model $\boldsymbol{\Theta}$ with weight $\mathcal{W}$, constraint Ω_0, total searching iterations T, population size P, sample size S
Output: Compressed model $\boldsymbol{\Theta}^*$

Initialize $\boldsymbol{\Theta}$ with pretrained parameters
Generate $\Psi(\mathcal{W}_l^i)$;
Initialize *pool* to a size of P
for t = 1 to T **do**
  Calculate the current mutation rate $\mathcal{R}_{mut}$ according to Eq.(7);
  $\mathcal{G}$ = **Agent** (S, *Pool*, $\mathcal{R}_{mut}$)
  According to Eq.(3) and Eq.(4), the **Director** gives the mask $\mathcal{M}$ under Ω_0;
  The agent works out the $\mathcal{L}_{fitness}$ according to Eq.(6) and replaces the most aging gene;
end for
Compress the network based on the optimal mask $\mathcal{M}^*$ and fine-tune it, then save the model with the highest accuracy as the best model $\boldsymbol{\Theta}^*$;
return Best compressed model $\boldsymbol{\Theta}^*$

III. Experiments

In this section, we evaluate the proposed method on two benchmark datasets for remote sensing image classification and three widely adopted CNNs. We first demonstrate the experimental setup in Section III-A, including an introduction to the datasets and networks, implementation details, and evaluation criteria. In Sections III-B and III-C, we report the performance of our algorithm on the two datasets in detail, respectively. In addition, some experiments are conducted for comparison with state of the arts (SOTAs). In Section III-D, we conduct ablation experiments. The settings in the proposed compression framework are further explored to find the most optimal configuration for the task.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 22:05:33 UTC from IEEE Xplore. Restrictions apply.

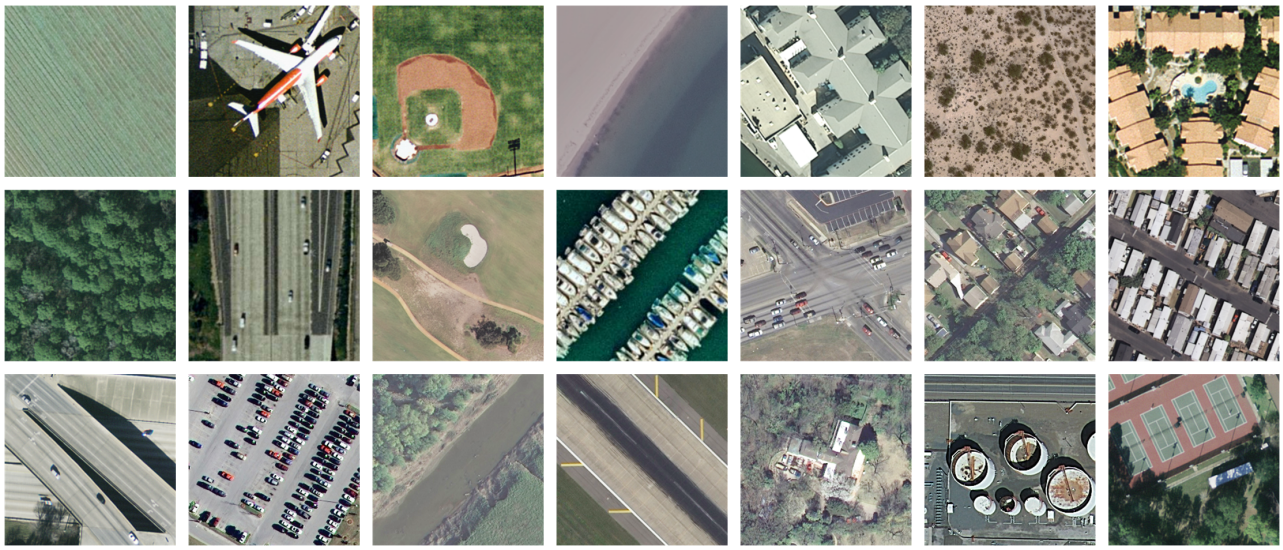

Fig. 4. Example images from the UCML-21 dataset containing 21 land-use scene categories: agricultural, airplane, baseball diamond, beach, buildings, chaparral, dense residential, forest, freeway, golf course, harbor, intersection, medium residential, mobile homepark, overpass, parking lot, river, runway, sparse residential, storage tanks, and tennis court.

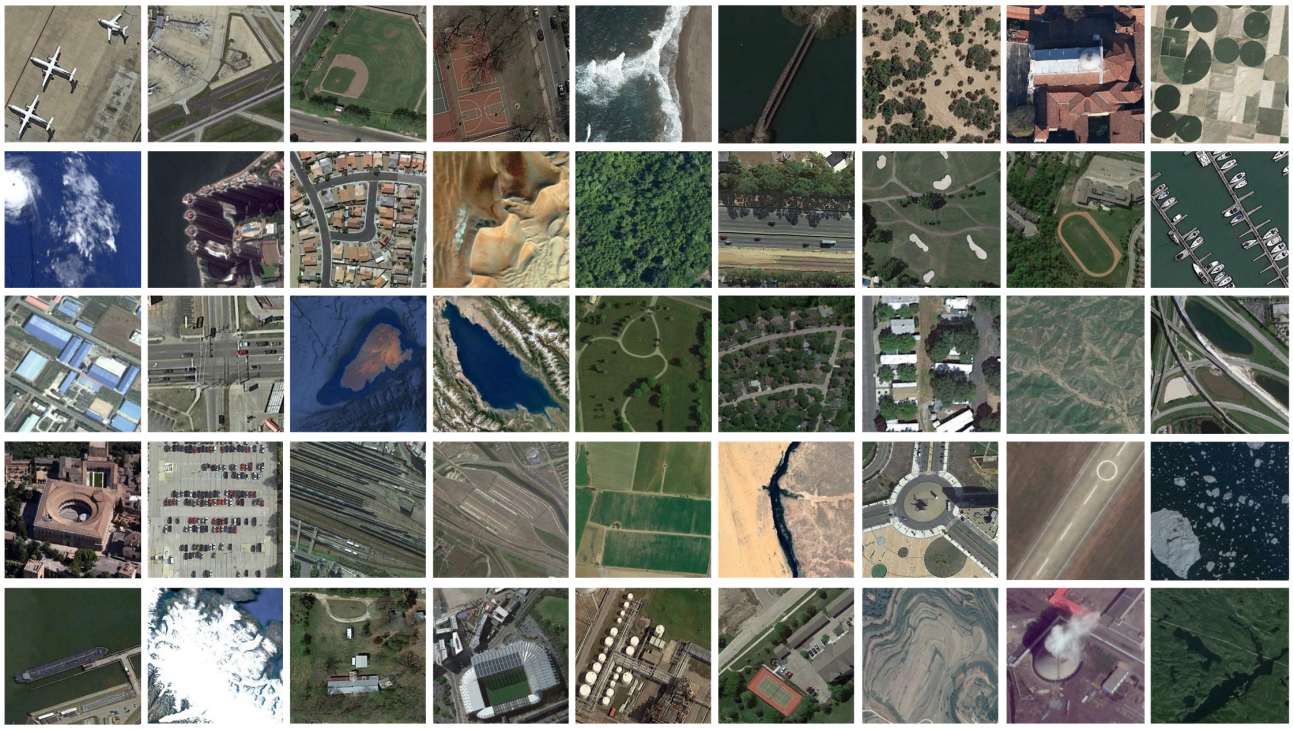

Fig. 5. Example images from the NWPU-45 dataset containing 45 land-use scene categories: airplane, airport, baseball diamond, basketball court, beach, bridge, chaparral, church, circular farmland, cloud, commercial area, dense residential, desert, forest, freeway, golf course, ground track field, harbor, industrial area, intersection, island, lake, meadow, medium residential, mobile home park, mountain, overpass, palace, parking lot, railway, railway station, rectangular farmland, river, roundabout, runway, sea ice, ship, snowberg, sparse residential, stadium, storage tank, tennis court, terrace, thermal power station, and wetland.

A. Experiment Setup

1) Datasets and Models: We conduct extensive experiments on two publicly available benchmarks for REmote Sensing Image Scene Classification (RESISC), UC Merced land-use dataset and NWPU-RESISC45 dataset, which are abbreviated as UCML-21 [36] and NWPU-45 [37], respectively. The first one is the UCML-21, which consists of images of 21 land-use scene categories selected from aerial ortho imagery. Each class contains 100 images with a size of 256 × 256 pixels and a 1-ft spatial resolution. Fig. 4 shows some example images randomly selected from UCML-21. The other one is the NWPU-45, which is extracted from Google Earth (Google Inc.) and covers more than 100 countries and regions all over the world. The NWPU-45 contains 31 500 images, containing 45 scene classes with 700 images in each class. Every image is 256 × 256 pixels in the red–green–blue (RGB) color space. The spatial resolution varies from about 30 to

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 22:05:33 UTC from IEEE Xplore. Restrictions apply.

0.2 m each pixel for most of the scene classes. Compared with UCML-21, NWPU-45 not merely covers most categories of UCML-21, such as airplane, baseball diamond, forest, freeway, intersection, and storage tank, but also includes more generalized scene classification data, such as commercial area, mountain, church, palace, cloud, and sea ice, making classification task and network pruning more challenging. Fig. 5 shows some example images from NWPU-45 with one sample each class.

To demonstrate the broad applicability of our approach, our method was tested on CNNs with three different structures. They are VGG-16 [26], VGG-19 [26], and ResNet-56 [27]. The number of convolutional layers is 13, 16, and 55, respectively, with VGG-19 having three fully connected (FC) layers, while VGG-16 and ResNet-56 have only one FC layer. Meanwhile, the inference of VGG-19 occupies the highest memory space.

2) Evaluation Criteria: In our experiments, classification performance is evaluated by overall accuracy, while FLOPs, parameters (Paras), and BOPs are three criteria employed to evaluate computational complexity.

As for accuracy evaluation, we demonstrate the Top-1 accuracy of the pruned models for UCML-21 and NWPU-45.

For a convolution layer converting C_i channels into C_o channels, FLOPs and Paras are

$$\begin{aligned} \text{FLOPs} &= H_o W_o \left(C_i 2K^2 + 1\right) C_o \\ \text{Paras} &= \left(C_i K^2 + 1\right) C_o \end{aligned} \tag{11}$$

where H_o and W_o define the output spatial size, and K is the kernel size. When the bias is none, (11) turns out to be

$$\begin{aligned} \text{FLOPs} &= H_o W_o \left(C_i 2K^2\right) C_o \\ \text{Paras} &= \left(C_i K^2\right) C_o. \end{aligned} \tag{12}$$

For a C_o-channel output, BN involves two channel-wise parameters, i.e., learned scaling factor and bias term. The FLOPs and Paras of BN are calculated as

$$\begin{aligned} \text{FLOPs} &= 2H_o W_o C_o \\ \text{Paras} &= 2C_o. \end{aligned} \tag{13}$$

The linear layer is indispensable for classification contributing to the transformation between latent space and label space. The FLOPs and Paras of the linear layer are calculated as

$$\begin{aligned} \text{FLOPs} &= C_i C_o \\ \text{Paras} &= C_i C_o + C_o. \end{aligned} \tag{14}$$

The BOPs metric has been used by several papers [34], [38], [39]. The BOP for a convolution layer converting C_i channels into C_o channels is defined as

$$\text{BOPs} = C_i C_o K^2 H_o W_o B_W B_A \tag{15}$$

where B_W and B_A denote weight and activation bitwidth of current layer. The BOPs compression ratio is defined as the ratio between the total BOPs of the uncompressed and compressed models.

Without loss of generality, we use the FLOPs decline rate to measure only the pruning effect and use the BOP compression ratio to measure the overall effect from pruning and quantization. As seen from (15), BOPs count is a function of both number of channels remaining after pruning and quantization bitwidth; hence, BOPs compression ratio is a suitable metric to measure a CNN's overall compression.

3) Software Environment and Implementation Details: The proposed framework is implemented with PyTorch on an NVIDIA A100-SXM4-80 GB.

All the networks are optimized by a stochastic gradient descent (SGD) algorithm [43] for 300 epochs. The momentum is 0.9, and the initial learning rate is 0.001, which decays by ten times at epochs 150 and 225. The batch size is 16 and 64 for UCML-21 and NWPU-45, respectively. For all experiments, random crop and horizontal flip provided by the official PyTorch serve for the standard data augmentation.

We chose Rank as the filter property to learn the filter importance score, and the reason for this choice will be given in subsequent experiments. As explored in HRank [31], a small batch of input images can be used to accurately estimate the expectation of the feature map rank. Thus, for all benchmarks and architectures, we randomly sample 768 images to estimate the average Rank of each feature map.

In the course of our agent searching, the population size P, the number of genes sampled S, and the total number of iterations T are fixed at 64, 16, and 400, respectively. β in $\mathcal{L}_{\text{fitness}}$ is $1e^{-10}$. Adjustable constants α and k in the progressive shrinkage mutation rate are set to 20 and 0.05, while λ is 1.25. In the course of our director assigning, τ_1, τ_2, and $\Delta\tau$ are set to 0.1, 100, and 5 for ResNet-56 and VGG-19 as well as 0.1, 10, and 5 for VGG-16 (specifically, in the ablation experiment, we modify τ_1, τ_2, and $\Delta\tau$ to 0.2, 2.2, and 1 when calculating the $L2$-norm on VGG-16 in order to better fit the filter score distribution). The safety region is 10% for all layers in all the experiments presented.

Following the experience of [44] and [45], an iterative fine-tuning strategy was used for the fine-tuning phase, which resulted in better recovery of accuracy. In the first step, the full accuracy network is fine-tuned after pruning only. In the second step, the best weight from the first step is inherited, and a pruned network with mixed precision activation quantization and real-valued weight is trained. In the third step, the best weight from the second step is inherited, and mixed precision quantization is applied to both weight and activation, and then, the pruned and quantized network is fine-tuned.

We use our method on the UCML-21 and NWPU-45 datasets with 21 and 45 land-use scenes, respectively. The images in both UCML-21 and NWPU-45 are divided into a training/fine-tuning set and a validation/test set in a 4:1 ratio. The output feature maps take up very little space due to their small size. Although the FC layer takes up a relatively large proportion of the parameters, we only compress the convolutional layer, considering that inference efficiency is the highest criterion for network compression. It is also worth noting that we have retained the structure of the original VGG-19, which has a larger number of parameters in FC layers. However, we have simplified the FC layers of VGG-16 by using only one linear layer. This is why VGG-19 has a lower rate of decline in the number of parameters.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 22:05:33 UTC from IEEE Xplore. Restrictions apply.

TABLE I

RESULTS OF OUR METHOD ON THE UCML-21 DATASET

Network	Top-1 Acc.(%)	FLOPs(G)	FLOPs ↓(%)	Param.(M)	Param. ↓(%)	BOPs(G)	BOPs Comp.
VGG-16	94.520	40.088	\	14.983	\	20526.080	\
Compressed VGG-16	**95.476**	**20.012**	**50.08**	**3.429**	**77.12**	**512.575**	**40.04**
VGG-19	96.430	51.200	\	139.643	\	26214.400	\
Compressed VGG-19	**95.238**	**25.588**	**50.07**	**125.626**	**10.04**	**654.827**	**40.03**
ResNet-56	92.620	16.062	\	0.850	\	8223.744	\
Compressed ResNet-56	**93.095**	**8.006**	**50.16**	**0.302**	**64.51**	**205.586**	**40.00**

TABLE II

COMPARISONS OF OUR METHOD ON THE UCML-21 DATASET

Network	Method	Top-1 Accuracy Drop(%)	FLOPs Decline Rate(%)	Param. Decline Rate(%)	BOPs Comp.
VGG-16	interpretable [40]	0.280	45.45	34.52	1.83
	Guo.et.al [25]	0.210	47.22	36.27	1.89
	uniform pruning	0.820	47.04	29.50	1.89
	L1-norm-based pruning [41]	1.970	38.04	45.28	1.61
	HRank [31]	0.472	50.00	\	2.00
	LeGR [35]	-0.480	50.17	18.69	2.01
	CC-SG	**-0.956**	**50.08**	**77.12**	**40.04**
VGG-19	Guo.et.al [25]	0.520	48.52	36.79	1.94
	uniform pruning	1.950	48.47	30.74	1.94
	network sliming [42]	0.570	50.98	51.04	2.04
	CC-SG	**1.190**	**50.07**	**10.04**	**40.03**
ResNet-56	uniform pruning	1.670	45.68	47.58	1.84
	HRank [31]	0.239	50.00	\	2.00
	LeGR [35]	4.760	51.64	69.27	2.07
	CC-SG	**-0.475**	**50.16**	**64.51**	**40.00**

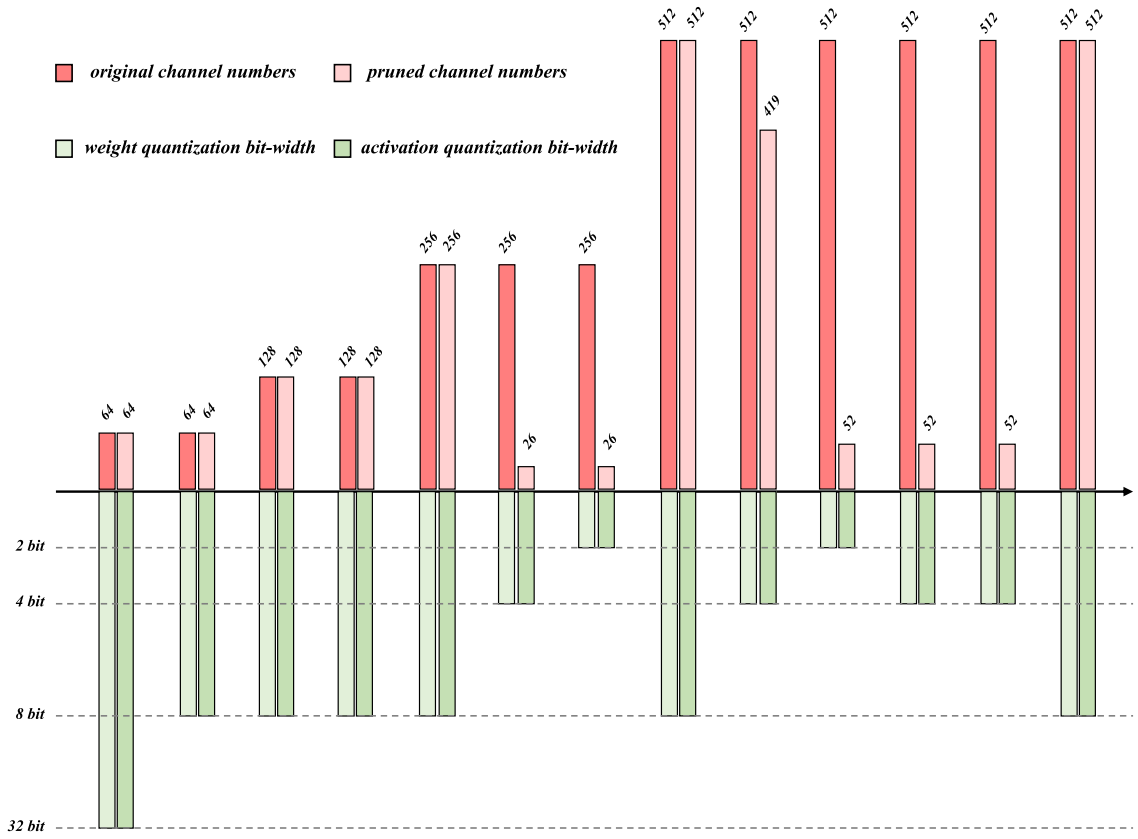

Fig. 6. Situation of pruning and quantization of the VGG-16's each layer on the UCML-21 dataset.

B. Experiments and Comparisons on the UCML-21 Dataset

Detailed experimental results, including overall accuracy, FLOPs, and the number of parameters and BOPs for different types of networks before and after compression, are given in Table I. It can be seen that with a small loss of accuracy, all networks are able to reduce their computational cost by pruning certain channels and quantizing weights and activation

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 22:05:33 UTC from IEEE Xplore. Restrictions apply.

TABLE III
RESULTS OF OUR METHOD ON THE NWPU-45 DATASET

Network	Top-1 Acc.(%)	FLOPs(G)	FLOPs ↓(%)	Param.(M)	Param. ↓(%)	BOPs(G)	BOPs Comp.
VGG-16	91.840	40.088	\	14.996	\	20526.080	\
Compressed VGG-16	**91.365**	**20.016**	**50.07**	**3.370**	**77.53**	**511.884**	**40.10**
VGG-19	91.980	51.200	\	139.741	\	26214.400	\
Compressed VGG-19	**92.984**	**25.780**	**49.65**	**125.510**	**10.18**	**655.328**	**40.00**
ResNet-56	87.210	16.062	\	0.851	\	8223.744	\
Compressed ResNet-56	**87.413**	**7.854**	**51.10**	**0.377**	**55.69**	**205.568**	**40.00**

TABLE IV
COMPARISONS OF OUR METHOD ON THE NWPU-45 DATASET

Network	Method	Top-1 Accuracy Drop(%)	FLOPs Decline Rate(%)	Param. Decline Rate(%)	BOPs Comp.
VGG-16	Guo.et.al [25]	0.230	42.77	28.51	1.75
	uniform pruning	0.730	42.68	26.43	1.74
	L1-norm-based pruning [41]	1.470	38.04	45.25	1.61
	LeGR [35]	0.475	50.64	22.37	2.03
	CC-SG	**0.475**	**50.07**	**77.53**	**40.10**
VGG-19	Guo.et.al [25]	0.130	44.20	29.37	1.79
	uniform pruning	1.370	44.08	27.61	1.79
	network sliming [42]	0.110	46.58	55.18	1.87
	CC-SG	**-1.004**	**49.65**	**10.18**	**40.00**
ResNet-56	uniform pruning	3.770	45.68	47.54	1.84
	LeGR [35]	3.050	50.54	63.39	2.02
	CC-SG	**-0.203**	**51.10**	**55.69**	**40.00**

functions. Even we obtained higher overall accuracies than the uncompressed networks on ResNet-56 and VGG-16. The feature extraction network structures of VGG-16 and VGG-19 are similar, differing by three convolutional layers, but due to the different design of the FC layers, VGG-16 performs more prominently on the relatively small dataset of UCML-21, whereas the parameters of VGG-19 have a relatively large amount of redundancy, and there is bound to be some loss in accuracy to achieve the same decline rate of FLOPs without compressing the FC layers.

One possible explanation we can deduce is that there are redundant connections between the networks, which can be pruned to achieve a more efficient network structure, and, more importantly, that the compressed model achieves hardware-friendliness and can be deployed directly on hardware devices. However, this benefit comes with the risk that accuracy may be compromised if more channels are pruned, which is a shortcoming of structured pruning compared to unstructured pruning [46].

Table II shows our experimental results and comparisons, where the rate of decrease in accuracy lies in the difference in accuracy between the uncompressed network and the compressed model. The $L1$-norm-based pruning [41] prunes a half number of channels in conv layers 8–13. The hyperparameter that controls the trade-off of network sliming [42] is set as $10e^{-4}$. As can be seen in Table II, the proposed method outperforms the majority of the listed comparison algorithms based on not only pruning but also quantization, which validates the effectiveness and feasibility of the pruned-quantization joint compression framework and the EEA agent.

We also plot in Fig. 6 the situation of pruning and quantization of the VGG-16's each layer on the UCML-21 dataset after compression. As can be seen in the figure, several layers are untrimmed, which suggests that they are particularly sensitive layers and critical to classification performance.

TABLE V
RESULTS OF DIFFERENT $\Psi(\mathcal{W}_l^i)$ VALUES ON THE VGG-16 ON THE UCML-21

Choice	Top-1 Acc. ↓(%)	FLOPs ↓(%)	Param. ↓(%)	BOPs Comp.
L1-norm	-0.481	50.75	26.29	40.04
L2-norm	-0.361	50.85	62.51	40.01
Rank	**-0.956**	**50.08**	**77.12**	**40.04**

C. Experiments and Comparisons on the NWPU-45 Dataset

Table III shows the results of our method on the NWPU-45 dataset. Due to the increase in scene class size, the classification performance of each network generally decreases compared with the results on the UCML-21 dataset, and the difficulty of compression increases. However, when it comes to the NWPU-45 dataset, VGG-19 dominates by virtue of its large number of parameters compared with the simplified VGG-16, and the effectiveness of our algorithm on VGG-19 is brought into play.

Table IV summarizes our experimental results with compared methods on the NWPU-45 dataset. The performance of ours still outperforms uniform pruning and LeGR obviously. As for $L1$-norm-based pruning, because more middle and higher layers that account for a large proportion of parameters are removed, it cuts down more percentage of parameters. Even so, we reduce computational costs and drop much less accuracy than it.

D. Ablation Study

1) Choice of Filter Property: As mentioned above, $\Psi(\mathcal{W}_l^i)$ in the filter score can be $L1$-norm, $L2$-norm, Rank, or other filter properties derived from filter weight or feature map. In our actual experiments, we would like to choose a relatively stable criterion for calculating filter scores, so we conduct three separate sets of experiments on the VGG-16 network on the UCML-21 dataset under similar resource constraints and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 22:05:33 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
EFFECT OF PROGRESSIVE SHRINKAGE MUTATION RATE FOR COMPRESSING VGG-16 AND RESNET-56 ON UCML-21

Network	Mutation Rate	Top-1 Acc. ↓(%)	FLOPs ↓(%)	Param. ↓(%)	BOPs Comp.
VGG-16	Fixed	-0.242	50.11	70.80	40.01
	Progressive Shrinkage	**-0.956**	**50.08**	**77.12**	**40.04**
ResNet-56	Fixed	0.239	51.63	75.11	40.07
	Progressive Shrinkage	**-0.475**	**50.16**	**64.51**	**40.00**

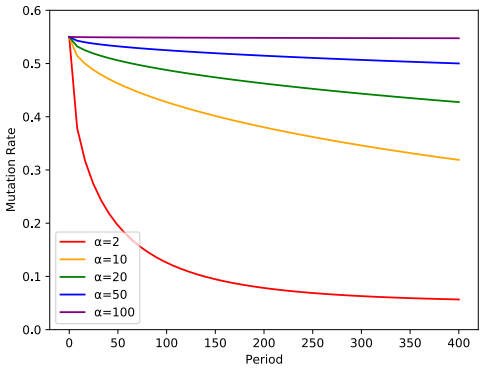

Fig. 7. Progressive shrinkage mutation rate curve at various α values.

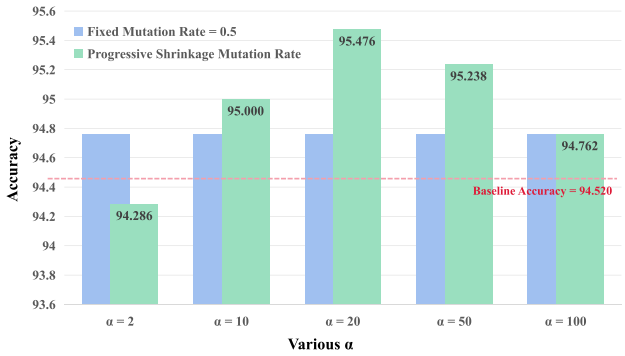

Fig. 8. Compressing VGG-16 on the UCML-21 dataset with progressive shrinkage mutation rate curve at various α.

the same experimental configuration to explore the relatively better $\Psi(\mathcal{W}_l^i)$.

From Table V we can see that Rank performs better, so in our other experiments, Rank is used as the filter property of choice. It is worth mentioning that it has been observed in HRank [31] that the Rank generated by a single filter is robust to the input images, so that a small batch of input images can be used to accurately estimate the expected value of the feature maps. In addition, feature maps are an intermediate step that reflects both the properties of the filter and the input images, and they also capture the transformation of the input images through the layers.

Of course, $L1$-norm and $L2$-norm as properties of the filter also reflect the importance of the filter within its layer and are easier and faster to calculate when compared to Rank, so users can choose according to their actual needs.

2) Fixed Mutation Rate and Progressive Shrinkage Mutation Rate: As shown in Table VI, we run experiments on both VGG-16 and ResNet-56 networks on the UCML-21 dataset, comparing the use of fixed mutation rate 0.1 and progressive shrinkage mutation rate ($\alpha = 20$ and $k = 0.05$), respectively, during agent exploration, and it is clear that progressive shrinkage mutation rate is an effective improvement that can help the agent find relatively superior compression configurations more efficiently.

3) Various α Values in Progressive Shrinkage Mutation Rate: Fig. 7 plots the progressive shrinkage mutation rate curve at various α values. After this session determined the use of progressive shrinkage mutation rates, we want to explore the most appropriate α. As for k, we fix it at 0.05, because its function is to ensure that the mutation rate does not end up contracting to 0 but gradually approaches a value, and we find that it was sufficient to take a value of k that was slightly greater than 0 without any particular study.

Fig. 8 gives the results of this set of exploratory experiments, where we conduct experiments with similar compression multiples and the same experimental setup for $\alpha = 2$, 10, 20, 50, and 100, respectively. In addition, to demonstrate that it is not the overall higher mutation rate that causes less accuracy loss after compression, we have added an experiment with a fixed mutation rate of 0.5 for similar compression multiples and the same experimental setup. In summary, we can further demonstrate the validity of the progressive shrinkage mutation rate and the fact that $\alpha = 20$ is a relatively well-suited parameter for our task.

IV. CONCLUSION

We propose a novel pruning-quantization co-compression method for remote sensing image classification, called CC-SG, which consists of two members, an agent and a director. The agent is an EEA for searching for "superior gene," who introduces gene age and progressive shrinkage mutation rate. Immediately following, a compression mask and feedback from the director to the agent, who computes the fitness function and completes an update. At the end of the search iteration, the agent generates the "superior gene," the director generates the optimal compression mask, and the model is finally lightened by compression and fine-tuning. Extensive experiments on UCML-21 and NWPU-45 datasets demonstrate the efficiency and effectiveness of our new perspective of model compression for remote sensing image classification.

REFERENCES

[1] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2021.
[2] D. Hong et al., "More diverse means better: Multimodal deep learning meets remote sensing imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 5, pp. 4340–4354, May 2020.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 22:05:33 UTC from IEEE Xplore. Restrictions apply.

[3] X. Wu, D. Hong, and J. Chanussot, "Convolutional neural networks for multimodal remote sensing data classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–10, 2021.

[4] Q. Zhu, Y. Zhong, B. Zhao, G.-S. Xia, and L. Zhang, "Bag-of-visual-words scene classifier with local and global features for high spatial resolution remote sensing imagery," *IEEE Geosci. Remote Sens. Lett.*, vol. 13, no. 6, pp. 747–751, Jun. 2016.

[5] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. (CVPR)*, vol. 1, Jun. 2005, pp. 886–893.

[6] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vis.*, vol. 60, no. 2, pp. 91–110, Feb. 2004.

[7] Q. Bi, K. Qin, H. Zhang, J. Xie, and K. Xu, "APDC-Net: Attention pooling-based convolutional network for aerial scene classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 9, pp. 1603–1607, Sep. 2020.

[8] R. Zhu, L. Yan, N. Mo, and Y. Liu, "Attention-based deep feature fusion for the scene classification of high-resolution remote sensing images," *Remote Sens.*, vol. 11, no. 17, p. 1996, Aug. 2019.

[9] W. Zhang, P. Tang, and L. Zhao, "Remote sensing image scene classification using CNN-CapsNet," *Remote Sens.*, vol. 11, no. 5, p. 494, 2019.

[10] K. Xu, H. Huang, Y. Li, and G. Shi, "Multilayer feature fusion network for scene classification in remote sensing," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 11, pp. 1894–1898, Nov. 2020.

[11] Y.-M. Zhang, C.-C. Lee, J.-W. Hsieh, and K.-C. Fan, "CSL-YOLO: A new lightweight object detection system for edge computing," 2021, *arXiv:2107.04829*.

[12] Z. Qin et al., "ThunderNet: Towards real-time generic object detection on mobile devices," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 6718–6727.

[13] Y. Li, S. Gu, C. Mayer, L. Van Gool, and R. Timofte, "Group sparsity: The Hinge between filter pruning and decomposition for network compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 8018–8027.

[14] B. Li et al., "Random sketch learning for deep neural networks in edge computing," *Nature Comput. Sci.*, vol. 1, no. 3, pp. 221–228, 2021.

[15] N. Jiang, X. Zhao, C. Zhao, Y. An, M. Tang, and J. Wang, "Pruning-aware sparse regularization for network pruning," 2022, *arXiv:2201.06776*.

[16] F. Jonathan and C. Michael, "The lottery ticket hypothesis: Training pruned neural networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019, pp. 1–42.

[17] Y. Zhong et al., "IntraQ: Learning synthetic images with intra-class heterogeneity for zero-shot network quantization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 12339–12348.

[18] Z. Wang, H. Xiao, J. Lu, and J. Zhou, "Generalizable mixed-precision quantization via attribution rank preservation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 5291–5300.

[19] S. Li et al., "Distilling a powerful student model via online knowledge distillation," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Mar. 7, 2022, doi: 10.1109/TNNLS.2022.3152732.

[20] J. Zhu et al., "Complementary relation contrastive distillation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 9260–9269.

[21] X. Ding, G. Ding, Y. Guo, and J. Han, "Centripetal SGD for pruning very deep convolutional networks with complicated structure," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 4943–4953.

[22] Y. He, P. Liu, Z. Wang, Z. Hu, and Y. Yang, "Filter pruning via geometric median for deep convolutional neural networks acceleration," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 4340–4349.

[23] Y. He, G. Kang, X. Dong, Y. Fu, and Y. Yang, "Soft filter pruning for accelerating deep convolutional neural networks," 2018, *arXiv:1808.06866*.

[24] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," 2015, *arXiv:1510.00149*.

[25] X. Guo, B. Hou, B. Ren, Z. Ren, and L. Jiao, "Network pruning for remote sensing images classification based on interpretable CNNs," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–15, 2021.

[26] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.

[27] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[28] W. Tang, X. Wei, and B. Li, "Automated model compression by jointly applied pruning and quantization," 2020, *arXiv:2011.06231*.

[29] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2019, vol. 33, no. 1, pp. 4780–4789.

[30] X. Zhang, W. Xie, Y. Li, J. Lei, and Q. Du, "Filter pruning via learned representation median in the frequency domain," *IEEE Trans. Cybern.*, early access, Nov. 19, 2021, doi: 10.1109/TCYB.2021.3124284.

[31] M. Lin et al., "HRank: Filter pruning using high-rank feature map," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 1529–1538.

[32] K. Xu, D. Zhang, J. An, L. Liu, L. Liu, and D. Wang, "GenExp: Multi-objective pruning for deep neural network based on genetic algorithm," *Neurocomputing*, vol. 451, pp. 81–94, Sep. 2021.

[33] Y. Sui, M. Yin, Y. Xie, H. Phan, S. Aliari Zonouz, and B. Yuan, "CHIP: Channel independence-based pruning for compact neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 24604–24616.

[34] Y. Wang, Y. Lu, and T. Blankevoort, "Differentiable joint pruning and quantization for hardware efficiency," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*. Berlin, Germany: Springer, 2020, pp. 259–277.

[35] T.-W. Chin, R. Ding, C. Zhang, and D. Marculescu, "Towards efficient model compression via learned global ranking," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 1518–1528.

[36] Y. Yang and S. Newsam, "Bag-of-visual-words and spatial extensions for land-use classification," in *Proc. 18th SIGSPATIAL Int. Conf. Adv. Geographic Inf. Syst.*, Nov. 2010, pp. 270–279.

[37] G. Cheng, J. Han, and X. Lu, "Remote sensing image scene classification: Benchmark and state of the art," *Proc. IEEE*, vol. 105, no. 10, pp. 1865–1883, Oct. 2017.

[38] J. Bethge, C. Bartz, H. Yang, Y. Chen, and C. Meinel, "MeliusNet: Can binary neural networks achieve MobileNet-level accuracy?" 2020, *arXiv:2001.05936*.

[39] C. Louizos, M. Reisser, T. Blankevoort, E. Gavves, and M. Welling, "Relaxed quantization for discretized neural networks," 2018, *arXiv:1810.01875*.

[40] Q. Zhang, Y. N. Wu, and S.-C. Zhu, "Interpretable convolutional neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 8827–8836.

[41] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. Peter Graf, "Pruning filters for efficient ConvNets," 2016, *arXiv:1608.08710*.

[42] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2736–2744.

[43] Y. E. Nesterov, "A method for solving the convex programming problem with convergence rate o $(1/k^2)$," in *Doklady Akademii Nauk SSSR*, vol. 269. Russia: Moskva, Nauka, 1983, pp. 543–547.

[44] B. Martinez, J. Yang, A. Bulat, and G. Tzimiropoulos, "Training binary neural networks with real-to-binary convolutions," 2020, *arXiv:2003.11535*.

[45] Z. Liu, Z. Shen, M. Savvides, and K.-T. Cheng, "ReActNet: Towards precise binary neural network with generalized activation functions," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*. Berlin, Germany: Springer, 2020, pp. 143–159.

[46] M. Lin et al., "1xN pattern for pruning convolutional neural networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, early access, Aug. 2, 2022, doi: 10.1109/TPAMI.2022.3195774.

Weiying Xie (Member, IEEE) received the B.S. degree in electronic information science and technology from the University of Jinan, Jinan, China, in 2011, the M.S. degree in communication and information systems from Lanzhou University, Lanzhou, China, in 2014, and the Ph.D. degree in communication and information systems from Xidian University, Xi'an, China, in 2017.

She is currently an Associate Professor with the State Key Laboratory of Integrated Services Networks, Xidian University. Her research interests include neural networks, machine learning, hyperspectral image processing, and high-performance computing.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 22:05:33 UTC from IEEE Xplore. Restrictions apply.

Xiaoyi Fan received the B.E. degree in telecommunications engineering from Xidian University, Xi'an, China, in 2021, where she is currently pursuing the M.S. degree with the Image Coding and Processing Center, State Key Laboratory of Integrated Services Networks.

Her research interests include remote sensing image processing and efficient deep learning.

Xin Zhang received the B.E. degree in telecommunications engineering from Xidian University, Xi'an, China, in 2019, where she is currently pursuing the Ph.D. degree with the Image Coding and Processing Center, State Key Laboratory of Integrated Services Networks.

Her research interests include remote sensing image processing and efficient deep learning.

Yunsong Li (Member, IEEE) received the M.S. degree in telecommunication and information systems and the Ph.D. degree in signal and information processing from Xidian University, Xi'an, China, in 1999 and 2002, respectively.

In 1999, he joined the School of Telecommunications Engineering, Xidian University, where he is currently a Professor. He is also the Director of the Image Coding and Processing Center, State Key Laboratory of Integrated Services Networks, Xidian University. His research interests include image and video processing and high-performance computing.

Min Sheng (Senior Member, IEEE) received the M.S. degree in telecommunication and information systems and the Ph.D. degree in signal and information processing from Xidian University, Xi'an, China, in 2000 and 2004, respectively.

From July 2015 to September 2015, she was a Senior Visiting Scholar with the University of Toronto, Toronto, ON, Canada, researching on heterogeneous wireless networks. In 2000, she joined the School of Telecommunications Engineering, Xidian University, where she is currently a Professor. She is also the Director of the State Key Laboratory of Integrated Services Networks, Xidian University. Her research interests include spatial information networks, mobile communication networks, heterogeneous network convergence, and wireless self-organizing networks.

Leyuan Fang (Senior Member, IEEE) received the Ph.D. degree from the College of Electrical and Information Engineering, Hunan University, Changsha, China, in 2015.

From September 2011 to September 2012, he was a Visiting Ph.D. Student with the Department of Ophthalmology, Duke University, Durham, NC, USA, supported by the China Scholarship Council. From August 2016 to September 2017, he was a Post-Doctoral Researcher with the Department of Biomedical Engineering, Duke University. He is currently a Professor with the College of Electrical and Information Engineering, Hunan University, and an Adjunct Researcher with the Peng Cheng Laboratory, Shenzhen, China. His research interests include sparse representation and multiresolution analysis in remote sensing and medical image processing.

Dr. Fang was a recipient of one Second-Grade National Award at the Nature and Science Progress of China in 2019. He is an Associate Editor of the IEEE TRANSACTIONS ON IMAGE PROCESSING, the IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING, the IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, and *Neurocomputing*.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 22:05:33 UTC from IEEE Xplore. Restrictions apply.